

5부: CLI를 활용한 크로스 플랫폼(윈도우/맥) 자동화 작업 구축 가이드

개발 생산성을 높이는 가장 좋은 방법은 **귀찮고 반복되는 작업을 컴퓨터가 스스로 하게 만드는 자동화(Automation)**입니다.

이번 세션에서는 1부에서 설치한 터미널 및 파이썬(Python) 환경을 활용하여, 윈도우(Windows)와 맥(macOS) 모두에서 매끄럽게 돌아가는 백업 및 정리 자동화 시스템을 만들고, OS별 스케줄러(스케줄 예약 도구)를 통해 특정 시간에 자동으로 실행되도록 설정하는 방법을 배웁니다.

1. 크로스 플랫폼 자동화 도구로 파이썬(Python)을 쓰는 이유

각 OS는 자신만의 명령어 스크립트 언어를 가지고 있습니다.

- macOS / Linux: Bash, Zsh (`.sh` 파일)
- Windows: Batch (`.bat`), PowerShell (`.ps1`)

윈도우와 맥은 터미널 명령어가 완전히 다르기 때문에(예: 파일 복사는 맥은 `cp`, 윈도우는 `copy`), 하나의 스크립트로 두 환경 모두에서 작동하게 하려면 **파이썬(Python)**을 활용하는 것이 가장 똑똑한 방법입니다. 파이썬의 표준 라이브러리(`os`, `shutil` 등)는 OS에 구애받지 않고 파일 시스템을 제어할 수 있도록 도와줍니다.

2. [실습] 영어 학습 앱 파일 자동 백업 스크립트 작성

4부에서 만든 `session4_english_app` 폴더의 데이터를 하루에 한 번씩 날짜별로 안전하게 다른 폴더에 저장(백업)해 주는 스크립트를 만들어 봅시다.

Step 1: 백업 스크립트 파일 생성

프로젝트 폴더 내에 `backup.py` 파일을 생성하고 아래 코드를 복사해서 붙여넣습니다.

```
import os
import shutil
from datetime import datetime

# 1. 경로 설정 (현재 스크립트 파일이 있는 위치 기준)
CURRENT_DIR = os.path.dirname(os.path.abspath(__file__))
SOURCE_DIR = os.path.join(CURRENT_DIR, "session4_english_app")
BACKUP_BASE_DIR = os.path.join(CURRENT_DIR, "backups")

# 2. 날짜별 백업 폴더명 생성 (예: backups/backup_20260621_1410)
timestamp = datetime.now().strftime("%Y%m%d_%H%M")
target_backup_dir = os.path.join(BACKUP_BASE_DIR, f"backup_{timestamp}")

def run_backup():
    # 원본 폴더가 존재하는지 확인
    if not os.path.exists(SOURCE_DIR):
        print(f"❌ [에러] 원본 폴더를 찾을 수 없습니다: {SOURCE_DIR}")
        return

    try:
        # 백업 기본 폴더가 없으면 생성
```

```

if not os.path.exists(BACKUP_BASE_DIR):
    os.makedirs(BACKUP_BASE_DIR)
    print(f"📁 백업 저장소 폴더를 생성했습니다: {BACKUP_BASE_DIR}")

# 폴더 전체 복사 (shutil.copytree 이용)
shutil.copytree(SOURCE_DIR, target_backup_dir)
print(f"🌟 [백업 완료] {SOURCE_DIR} -> {target_backup_dir}")

except Exception as e:
    print(f"❌ 백업 중 에러 발생: {e}")

if __name__ == "__main__":
    run_backup()

```

Step 2: 수동 실행 테스트

터미널을 열고 아래 명령어로 스크립트를 직접 실행해 봅니다.

- 명령어:

```
python backup.py
```

- 결과: 실행 성공 문구와 함께 프로젝트 폴더 내부에 `backups` 폴더가 생기고, 그 안에 날짜별 백업 폴더가 정상적으로 복사되었는지 확인합니다.

3. OS별 터미널 자동 스케줄러(Scheduler) 연동하기

이제 우리가 직접 터미널에 명령어를 치지 않아도, 컴퓨터가 알아서 매일 밤 9시(21:00)에 이 백업 스크립트를 실행하도록 설정해 봅시다.

🍏 macOS 사용자: cron (크론) 활용하기

맥이나 리눅스 계열의 가장 전통적이고 널리 쓰이는 백그라운드 스케줄러는 `** cron **`입니다.

1. 터미널을 열고 크론 설정 편집 창을 엽니다:

```
crontab -e
```

(참고: 텍스트 편집기인 `vi` 또는 `nano` 창이 열립니다.)

2. 맨 아래 줄에 아래 규칙을 한 줄 추가합니다. (경로는 본인 컴퓨터의 실제 절대 경로로 변경해야 합니다.)

```

0 21 * * * /usr/bin/python3 /Users/사용자이름/Codes/Teaching_Vibe_Coding/backup.py

```

- 해석: `0 21 * * *` = 매월, 매일, 매요일 21시 0분에 다음 명령어를 실행하라.
3. 저장하고 나옵니다. (`vi` 편집기 기준 `:wq` 입력 후 엔터)
 4. 현재 등록된 예약 작업 목록을 확인하려면 아래 명령어를 칩니다:

```
crontab -l
```

♥ Windows 사용자: **schtasks** (작업 스케줄러 CLI) 활용하기

윈도우는 자체적인 **작업 스케줄러(Task Scheduler)** 프로그램이 내장되어 있습니다. 터미널(PowerShell) 명령어 한 줄로 편리하게 예약을 등록할 수 있습니다.

1. PowerShell을 **관리자 권한**으로 켭니다.
2. 아래 명령어를 한 줄로 입력하고 엔터를 누릅니다. (경로는 본인 컴퓨터의 실제 절대 경로로 수정해 주세요.)

```
schtasks /create /tn "VibeEnglishBackup" /tr "python  
C:\Codes\Teaching_Vibe_Coding\backup.py" /sc daily /st 21:00
```

- o `schtasks /create` : 새로운 작업 예약을 만듭니다.
 - o `/tn "VibeEnglishBackup"` : 작업의 이름(Task Name)을 지정합니다.
 - o `/tr "..."` : 실행할 명령어(Task Run)를 입력합니다.
 - o `/sc daily` : 실행 주기(Schedule)를 매일(daily)로 설정합니다.
 - o `/st 21:00` : 매일 밤 9시(Start Time)에 실행하도록 지정합니다.
3. 등록된 작업을 취소(삭제)하고 싶다면 아래 명령어를 입력합니다:

```
schtasks /delete /tn "VibeEnglishBackup" /f
```

4. 자동화 환경 구축 시 주의사항 (트러블슈팅)

1. 상대 경로 대신 '**절대 경로**' 사용 필수:

- o 스케줄러가 백그라운드에서 스크립트를 돌릴 때는 사용자가 평소 작업하던 현재 경로(Cwd)가 적용되지 않습니다. 따라서 `python` 실행 경로 및 스크립트 내 경로, 그리고 스케줄러 인자값에는 반드시 ****절대 경로(Full Path)****를 적어주어야 에러가 나지 않습니다.
- o **맥 절대 경로 확인법**: 터미널에서 `pwd` 입력
- o **윈도우 절대 경로 확인법**: 파일 탐색기 주소창 클릭 후 복사

2. 백그라운드 실행 권한 (**macOS**):

- o macOS의 최신 버전(Catalina 이상)은 터미널이 개인 폴더에 접근하려면 보안 권한 허용이 필요할 수 있습니다. 백업이 실행되지 않는다면 시스템 설정 -> 개인정보 보호 및 보안 -> 전체 디스크 접근 권한 에서 `cron` 또는 터미널 권한이 켜져 있는지 확인해 주세요.